## Лабораторна робота № 7

**Тема**: Багатопоточне програмування в Java.

**Мета роботи:** практика роботи з потоками в Java.

### Хід роботи

**Завдання 2.** Створити клас, що розширює Thread:

• Створити клас MyThread, що розширює Thread.

• Перевизначити метод run(). У циклі for вивести на консоль повідомлення «Я люблю програмувати!!!» 100 разів.

• Створити екземпляр класу та запустити новий потік.

• Вивести ім'я створеного потоку, його пріорітет, превірити чи він живий, чи є потоком демоном.

• Змінити ім'я, пріорітет створеного потоку та вивести в консоль оновлені значення.

• Після завершення роботи створеного потоку (використати метод join()) вивести ім’я головного потоку, та його пріорітет.

• Відобразити в консолі, коли ваш потік буде в стані NEW, RUNNUNG, TERMINATED.

Лістинг файлу TASK_02/Main.java:

```
package TASK_02;

public class Main {
    public static void main(String[] args) {
        MyThread myThread = new MyThread("MyThread");
        System.out.println("Стан потоку перед запуском: " + myThread.getState());

        myThread.start();
        System.out.println("Ім'я потоку: " + myThread.getName());
        System.out.println("Пріоритет потоку: " + myThread.getPriority());
        System.out.println("Чи є потік живим: " + myThread.isAlive());
        System.out.println("Чи є потік демоном: " + myThread.isDaemon());
        System.out.println("Стан потоку після запуску: " + myThread.getState());
        myThread.setName("NewMyThread");
        myThread.setPriority(Thread.MAX_PRIORITY);
        System.out.println("Нове ім'я потоку: " + myThread.getName());
        System.out.println("Новий пріоритет потоку: " + myThread.getPriority());
```

```
        try {
            myThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Стан потоку після завершення: " +
myThread.getState());

        Thread mainThread = Thread.currentThread();
        System.out.println("Ім'я головного потоку: " + mainThread.getName());
        System.out.println("Пріоритет головного потоку: " +
mainThread.getPriority());
    }
}
```

Лістинг файлу MyThread.java:

```
package TASK_02;

class MyThread extends Thread {

    public MyThread(String name) {
        super(name);
    }

    @Override
    public void run() {
        for (int i = 0; i < 100; i++) {
            System.out.println("Я люблю програмувати!!!");
        }
    }
}
```

```
Я люблю програмувати!!!
Я люблю програмувати!!!
Стан потоку після завершення: TERMINATED
Ім'я головного потоку: main
Пріоритет головного потоку: 5
```

Мал.1. Результат роботи програми

**Завдання 3.** Створити клас, що реалізує інтерфейс Runnable для виводу в консоль чисел від 0 до 10000, що діляться на 10 без залишку:

• Створити клас MyRunnable, який реалізує інтерфейс Runnable. • Імплементувати метод run().

• Визначити умову, якщо потік хочуть перервати, то завершити роботу потоку та вивести повідомлення «Розрахунок завершено!!!»

• Створити три потоки, які виконують завдання друку значень.

• Використовуємо статичний метод Thread.sleep(), щоб зробити паузу на 2 секунди для головного потоку, а після цього викликати для створених потоків метод interrupt().

Лістинг файлу TASK_04/Main.java:

```
package TASK_03;

public class Main {
    public static void main(String[] args) {
        MyRunnable task = new MyRunnable();

        Thread thread = new Thread(task, "Thread-1");
        thread.start();
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        thread.interrupt();
    }
}
```

Лістинг файлу MyRunnable.java:

```
package TASK_03;
public class MyRunnable implements Runnable {

    @Override
    public void run() {
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println(Thread.currentThread().getName() + ": Потік було
перервано перед початком.");
            return;
        }

        for (int i = 0; i <= 10000; i++) {
            if (i % 10 == 0) {
                System.out.println(Thread.currentThread().getName() + ": " + i);
            }

            if (i == 9990 || Thread.currentThread().isInterrupted()) {
                System.out.println(Thread.currentThread().getName() + ": Розраху-
нок завершено!!!");
                return;
            }
        }
    }
}
```

```
Thread-1: 9970
Thread-1: 9980
Thread-1: 9990
Thread-1: Розрахунок завершено!!!
```

Мал.2. Результат роботи програми

**Завдання 4.** Створити клас, що реалізує інтерфейс Runnable для вививедення арифметичної прогресії від 1 до 100 з кроком 1:

• Створити клас, який реалізує інтерфейс Runnable.

• Створити об'єкт зі статичною змінною result для збереження значення арифметичної прогресії.

• Перевизначити метод run(). Створити цикл for. У циклі виводимо через пробіл значення змінної result. Та додаємо наступне значення до змінної result та чекаємо 0,2 секунду.

• Забезпечити коректну роботу використовуючи синхронізований метод. • Створити три потоки, які виконують завдання друку значень.

Лістинг файлу TASK_05/Main.java:

```
package TASK_04;

public class Main {
    public static void main(String[] args) {
        ArithmeticProgressionRunnable task = new ArithmeticProgressionRunnable();

        Thread thread1 = new Thread(task, "Thread-1");
        Thread thread2 = new Thread(task, "Thread-2");
        Thread thread3 = new Thread(task, "Thread-3");

        thread1.start();
        thread2.start();
        thread3.start();
    }
}
```

Лістинг файлу ArithmeticProgressionRunnable.java:

```
package TASK_04;

public class ArithmeticProgressionRunnable implements Runnable {
    private static int result = 1;
    private synchronized void printNextProgression() {
        if (result <= 100) {
            System.out.print(result + " ");
            result++;
        }
    }

    @Override
    public void run() {
        while (result <= 100) {
            printNextProgression();
            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + ": Потік пе-
рервано.");
                return;
            }
        }
```

```
    }
}
```

```
70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
```

Мал.3. Результат роботи програми

**Завдання 5.** Переробити 4 завдання використовуючи блок синхронізації.

Лістинг файлу TASK_03/Main.java:

```
package TASK_05;

public class Main {
    public static void main(String[] args) {
        ArithmeticProgressionRunnable task = new ArithmeticProgressionRunnable();

        Thread thread1 = new Thread(task, "Thread-1");
        Thread thread2 = new Thread(task, "Thread-2");
        Thread thread3 = new Thread(task, "Thread-3");

        thread1.start();
        thread2.start();
        thread3.start();
    }
}
```

Лістинг файлу ArithmeticProgressionRunnable.java:

```
package TASK_05;

public class ArithmeticProgressionRunnable implements Runnable {
    private static int result = 1;

    @Override
    public void run() {
        while (result <= 100) {
            synchronized (ArithmeticProgressionRunnable.class) {
                if (result <= 101) {
                    System.out.print(result + " ");
                    result++;
                }
            }

            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + ": Потік пе-
рервано.");
                return;
            }
        }
    }
}
```

```
0 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101
```

Мал.4. Результат роботи програми

**Завдання 6**. Створити два потоки Reader та Printer. Reader зчитує введені дані з консолі та записує в змінну. Після цього інформує потік Printer та засипає на 1 се-

кунду, а потік Reader виводить дотриманий рядок. І так повторюється знову, поки користувач не завершить роботу програми.

- Змінну треба використати як об'єкт для синхронізації.
- Тут необхідно використати wait() і notify().

Лістинг файлу Printer.java:

```
package TASK_06;

public class Printer implements Runnable {
    private final SharedData sharedData;

    public Printer(SharedData sharedData) {
        this.sharedData = sharedData;
    }

    @Override
    public void run() {
        while (true) {
            String data = sharedData.getData();

            if ("exit".equalsIgnoreCase(data)) {
                break;
            }

            System.out.println("Отримані дані: " + data);
        }
    }
}
```

Лістинг файлу Reader.java:

```
package TASK_06;
import java.util.Scanner;

public class Reader implements Runnable {
    private final SharedData sharedData;

    public Reader(SharedData sharedData) {
        this.sharedData = sharedData;
    }

    @Override
    public void run() {
        Scanner scanner = new Scanner(System.in);
        while (true) {
            System.out.print("Введіть рядок (або 'exit' для виходу): ");
            String input = scanner.nextLine();
            sharedData.setData(input);

            if ("exit".equalsIgnoreCase(input)) {
                break;
            }

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
```

```
        scanner.close();
    }
}
```

Лістинг файлу ReaderPrinterExample.java:

```
package TASK_06;

public class ReaderPrinterExample {
    public static void main(String[] args) {
        SharedData sharedData = new SharedData();

        Thread readerThread = new Thread(new Reader(sharedData));
        Thread printerThread = new Thread(new Printer(sharedData));

        readerThread.start();
        printerThread.start();
    }
}
```

Лістинг файлу SharedData.java:

```
package TASK_06;

public class SharedData {
    private String data;

    public synchronized void setData(String data) {
        this.data = data;
        notify();
    }

    public synchronized String getData() {
        try {
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        return data;
    }
}
```

```
Введіть рядок (або 'exit' для виходу): Senatus
Отримані дані: Senatus
Введіть рядок (або 'exit' для виходу): PopulusQue
Отримані дані: PopulusQue
Введіть рядок (або 'exit' для виходу): exit
```

Мал.5. Результат роботи програми

**Завдання 7.** Створити програму для знаходження суми цифр в масиві на 1 000 000 елементів:

- Заповнити масив числами використовуючи клас Random.
- Реалізувати задачу в однопоточному та багатопоточному середовищі.

• Для багатопоточного середовища використати ExecutorService на 5 потоків та об'єкти потоків, що імплементують інтерфейси Runnable або Callable.

• Заміряти час виконання обох варіантів завдання використовуючи System.currentTimeMillis() та вивести результати в консоль.

Лістинг файлу TASK_07/Main.java:

```
package TASK_07;
import java.util.Random;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.List;
import java.util.ArrayList;

public class Main {
    private static final int ARRAY_SIZE = 1_000_000;
    private static final int THREAD_COUNT = 5;

    public static void main(String[] args) throws Exception {
        int[] array = new int[ARRAY_SIZE];
        Random random = new Random();

        for (int i = 0; i < ARRAY_SIZE; i++) {
            array[i] = random.nextInt(1000);
        }

        long startTime = System.currentTimeMillis();
        int singleThreadSum = calculateSumSingleThread(array);
        long singleThreadDuration = System.currentTimeMillis() - startTime;

        System.out.println("Сума (однопотокова): " + singleThreadSum);
        System.out.println("Час виконання (однопотоковий): " + singleThreadDura-
tion + " ms");

        startTime = System.currentTimeMillis();
        int multiThreadSum = calculateSumMultiThread(array);
        long multiThreadDuration = System.currentTimeMillis() - startTime;

        System.out.println("Сума (багатопотокова): " + multiThreadSum);
        System.out.println("Час виконання (багатопотоковий): " + multiThreadDura-
tion + " ms");
    }

    private static int calculateSumSingleThread(int[] array) {
        int sum = 0;
        for (int num : array) {
            sum += sumOfDigits(num);
        }
        return sum;
    }

    private static int calculateSumMultiThread(int[] array) throws Exception {
        ExecutorService executorService = Execu-
tors.newFixedThreadPool(THREAD_COUNT);
        List<Future<Integer>> futures = new ArrayList<>();

        int partSize = ARRAY_SIZE / THREAD_COUNT;
        for (int i = 0; i < THREAD_COUNT; i++) {
```

```
            int start = i * partSize;
            int end = (i == THREAD_COUNT - 1) ? ARRAY_SIZE : (i + 1) * partSize;

            Callable<Integer> task = new SumTask(array, start, end);
            futures.add(executorService.submit(task));
        }

        int totalSum = 0;
        for (Future<Integer> future : futures) {
            totalSum += future.get();
        }

        executorService.shutdown();
        return totalSum;
    }

    private static int sumOfDigits(int number) {
        int sum = 0;
        while (number != 0) {
            sum += number % 10;
            number /= 10;
        }
        return sum;
    }
}
```

Лістинг файлу Printer.java:

```
package TASK_07;
import java.util.concurrent.Callable;

public class SumTask implements Callable<Integer> {
    private final int[] array;
    private final int start;
    private final int end;

    public SumTask(int[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    public Integer call() {
        int sum = 0;
        for (int i = start; i < end; i++) {
            sum += sumOfDigits(array[i]);
        }
        return sum;
    }

    private int sumOfDigits(int number) {
        int sum = 0;
        while (number != 0) {
            sum += number % 10;
            number /= 10;
        }
        return sum;
    }
}
```

```
Сума (однопотокова): 13500213
Час виконання (однопотоковий): 5 ms
Сума (багатопотокова): 13500213
Час виконання (багатопотоковий): 22 ms
```

Мал.6. Результат роботи програми

**Посилання на репозиторій: https://github.com/VadymLeus/Y3S1-Java**

**Висновки**: я навчився працювати з потоками в Java.